

Predictive analytics

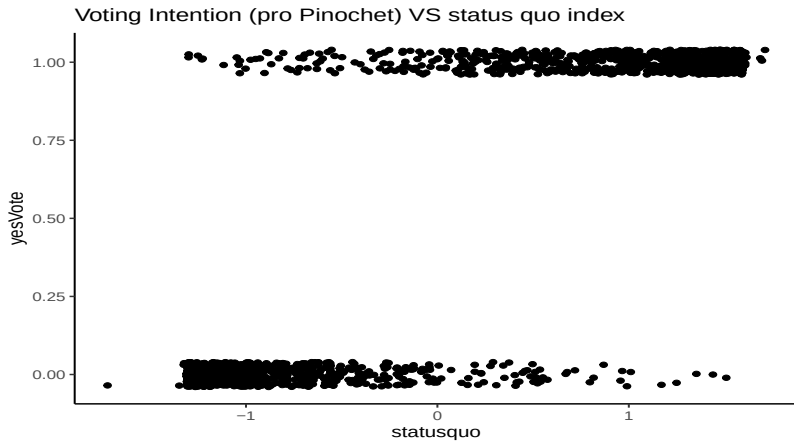
Block 3 - GLMs

Ca' Foscari University of Venice

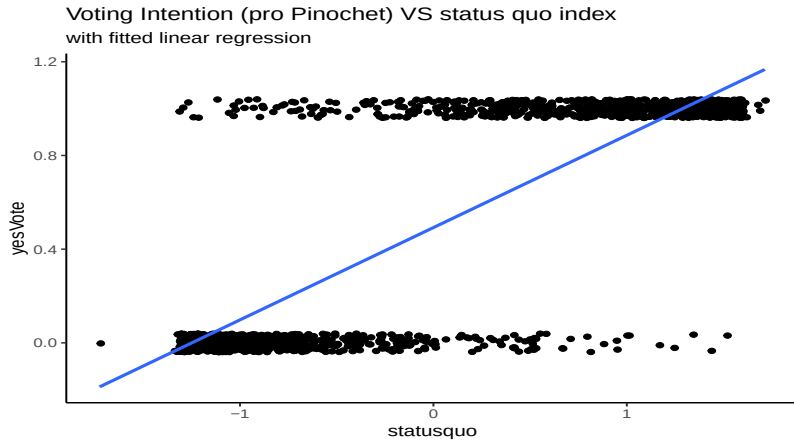
Academic year 2024/2025

Generalized linear models

A non-gaussian world I



A non-gaussian world II



The Generalized Linear Model (GLM) I

- In the Gaussian linear regression model

$$(Y \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}) \sim \mathcal{N}(\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}, \sigma^2)$$

- Under this assumption

$$\mathbb{E}[Y \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}] = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

- Two modifications of this situation
 - We'll allow for other distributions different from the normal
 - Instead of the conditional mean being a linear combination of the predictors, it can be some function of a linear combination of the predictors.

The Generalized Linear Model (GLM) II

- A **generalized linear model** has two parts:
 - a distribution of the response conditioned on the predictors. (This distribution needs to be from the **exponential family** of distributions.)
 - a **link** function, $g()$, that defines how the **linear combination** of the $p - 1$ predictors,

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_{p-1} x_{p-1}$$

is related to the mean of the response conditioned on the predictors, i.e.

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

Binary response and logistic regression I

- Categorical variables with two classes such as yes/no, cat/dog, sick/healthy, etc. can be coded in a binary variable, Y , using 0 and 1.

$$Y = \begin{cases} 1 & \text{yes} \\ 0 & \text{no} \end{cases}$$

- With a binary (Bernoulli) response, we'll mostly focus on the case when $Y = 1$, since we can obtain probabilities of $Y = 0$ with:

$$\Pr[Y = 0] = 1 - \Pr[Y = 1] = 1 - p$$

- Moreover

$$\mathbb{E}[Y] = \Pr[Y = 1] = p$$

Binary response and logistic regression II

Probability odd

$$\text{odd} = \frac{\Pr[Y = 1]}{\Pr[Y = 0]} = \frac{p}{1 - p}$$

- Interpretation: the odd is the probability for a positive event ($Y = 1$) divided by the probability of a negative event ($Y = 0$).
- When the odd is 1, the two events have equal probability. Odds greater than 1 favor a positive event. Odds smaller than 1 favor a negative event.
- The log odd is the **logit** transform applied to p .

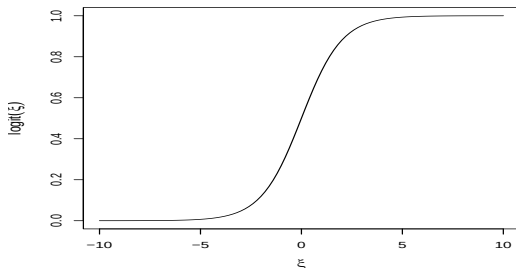
$$\text{logit}(\xi) = \log\left(\frac{\xi}{1 - \xi}\right)$$

- Since $p \in (0, 1)$, $\text{logit}(p) \in (-\infty, \infty)$.

Binary response and logistic regression III

- The inverse logit, also known as the **logistic** or **sigmoid** function is

$$\text{logit}^{-1}(\xi) = \frac{e^{\xi}}{1 + e^{\xi}} = \frac{1}{1 + e^{-\xi}} - 1$$



Note that for $\xi \in (-\infty, \infty)$, the logistic takes values between 0 and 1.

Binary response and logistic regression IV

- We have a binary response Y : this is the variable we wish to model as function of a vector of $p - 1$ predictors $X_1, \dots, X_{p-1}$
- We denote

$$\begin{aligned} p(x_1, \dots, x_{p-1}) &= \Pr[Y = 1 \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}] \\ &= \mathbb{E}[Y \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}] \end{aligned}$$

Binary response and logistic regression V

- A **logistic regression** model for Y is a model such that the logarithm of the odd

$$\frac{p(x_1, \dots, x_{p-1})}{1 - p(x_1, \dots, x_{p-1})} = \frac{\Pr[Y = 1 \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}]}{\Pr[Y = 0 \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}]}$$

is a linear combination of the predictors

$$\log \left(\frac{p(x_1, \dots, x_{p-1})}{1 - p(x_1, \dots, x_{p-1})} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

i.e.

$$p(x_1, \dots, x_{p-1}) = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}}}$$

Binary response and logistic regression VI

- Then

$$\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}] = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}}}$$

- Note that the **link** function g such that

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

is the logit transformation $\text{logit}(\xi) = \log\left(\frac{\xi}{1 - \xi}\right)$

Poisson response and log-linear regression I

- Suppose we have count responses Y that we wish to model in terms of a vector of predictors $x_1, \dots, x_{p-1}$.
- Assume that Y is Poisson with mean $\mathbb{E}[Y] = \mu > 0$, then:

$$\Pr(Y = y) = \frac{\mu^y}{y!} \exp(-\mu)$$

- Model μ in terms of the predictors:

$$(Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}) \sim \text{Poisson}(\mu(x_1, \dots, x_{p-1})),$$

we need to specify the function $\mu(\cdot)$

- We use a linear combination of the x_i to form the linear predictor.

Poisson response and log-linear regression II

- Since we require that $\mu(\cdot) \geq 0$, we can ensure this by that is:

$$\mu(x_1, \dots, x_{p-1}) = \exp\{\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}\}$$

i.e.

$$\log(\mu(x_1, \dots, x_{p-1})) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

- The link function g

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

is the logarithm and the model is termed **log-linear** model

- This corresponds to the canonical link: it is a natural way to construct the model

Generalizing I

- For the Poisson and Bernoulli case we have defined $g(E[Y]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$ taking different functions $g(\cdot)$
- We then have

$$Y \sim \text{Bern}(g^{-1}(\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}))$$

and

$$Y \sim \text{Pois}(g^{-1}(\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}))$$

- Poisson and Bernoulli are 1-parameter distribution: the variance is related to the mean
- In the Normal distribution we also have a nuisance parameter σ^2
- The Normal, Bernoulli and Poisson all belong to the Exponential Family of distribution: GLM generalizes the linear model to all distributions of the Exponential Family
- GLM is a broad class of models. We can use many different functions $g(\cdot)$: for each such function, we have a different GLM.

Exponential family I

- GLM assumes a specified parametric class for the conditional distribution of Y given $x_1, \dots, x_{p-1}$: the exponential family.
- This is formally defined as a parametric family whose probability density/mass function has the form

$$f(y; \theta, \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\}$$

where $a(\cdot)$, $b(\cdot)$, and $c(\cdot, \cdot)$ are specific functions.

- The parameter θ is called the **canonical parameter** and represents the location, while ϕ is called the **dispersion** parameter and represents the scale.
- Some nice properties

$$\mu = \mathbb{E}[Y] = b'(\theta), \quad \text{Var}[Y] = b''(\theta)a(\phi)$$

Exponential family II

- The **canonical link function** g is the one that transforms $\mathbb{E}[Y] = b'(\theta)$ into the **canonical parameter** θ . this happens if

$$\theta = g(\mathbb{E}[Y])$$

or if

$$\theta = b'^{-1}(\mathbb{E}[Y])$$

- The Variance of the distribution is also written as a function of θ (and ϕ)
- The $b''(\theta)$ function is called the *variance function*: specifies how the variance depends on the location parameter
- Some well-known examples of distributions belonging to the EF are given below.

Exponential family III

Gaussian distribution:

$$\begin{aligned} f(y; \mu, \sigma^2) &= \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{1}{2} \left(\frac{y - \mu}{\sigma} \right)^2 \right\} = \exp \left\{ -\frac{1}{2} \left(\frac{y - \mu}{\sigma} \right)^2 - \frac{1}{2} \log(\sqrt{2\pi\sigma^2}) \right\} \\ &= \exp \left\{ \frac{y\mu - 0.5\mu^2}{\sigma^2} - \frac{1}{2} \left(\frac{y^2}{\sigma^2} + \log(\sqrt{2\pi\sigma^2}) \right) \right\} = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\} \end{aligned}$$

where $\theta = \mu$, $\phi = \sigma^2$, $a(\phi) = \phi$, $b(\theta) = 0.5\theta^2$, and $c(y, \phi) = -\frac{1}{2} \left(\frac{y^2}{\phi} + \log(\sqrt{2\pi\phi}) \right)$

$$\mathbb{E}[Y] = b'(\theta) = \theta = \mu, \quad \text{Var}[Y] = b''(\theta)a(\phi) = \phi = \sigma^2$$

canonical link function $g(\xi) = \xi$, the identity function

Exponential family IV

Gamma distribution:

$$f(y; \lambda, \nu) = \frac{1}{\Gamma(\nu)} \lambda^\nu y^{\nu-1} e^{-\lambda y}$$

and we have $E[Y] = \nu/\lambda$ and $Var[Y] = \nu/\lambda^2$.

We can reparameterize the distribution taking $\mu = \nu/\lambda$, i.e. $\lambda = \nu/\mu$:

$$f(y; \mu, \nu) = \frac{1}{\Gamma(\nu)} \left(\frac{\nu}{\mu}\right)^\nu y^{\nu-1} e^{-\frac{\nu}{\mu} y}$$

so that

$$E[Y] = \mu \quad \text{and} \quad Var[Y] = \frac{\mu^2}{\nu}$$

Exponential family V

$$\begin{aligned}f(y; \mu, \nu) &= \frac{1}{\Gamma(\nu)} \left(\frac{\nu}{\mu}\right)^{\nu} y^{\nu-1} e^{-\nu y/\mu} \\&= \exp \{ -\log(\Gamma(\nu)) + \nu \log(\nu/\mu) + (\nu-1) \log(y) - \nu y/\mu \} \\&= \exp \left\{ \left(-\frac{1}{\mu} y - \log(\mu) \right) \nu - \log(\Gamma(\nu)) + \nu \log(\nu) + (\nu-1) \log(y) \right\} \\&= \exp \left\{ \frac{\theta y - b(\theta)}{a(\phi)} + c(y, \phi) \right\}\end{aligned}$$

where $\theta = -1/\mu$, $\phi = \nu$, $a(\phi) = 1/\nu$, $b(\theta) = \log(-1/\theta) = -\log(-\theta)$, and $c(y, \phi) = -\log(\Gamma(\nu)) + \nu \log(\nu) + (\nu-1) \log(y)$

$$\mathbb{E}[Y] = b'(\theta) = -1/\theta = \mu, \quad \text{Var}[Y] = b''(\theta)a(\phi) = \frac{1}{\theta^2} \frac{1}{\nu}$$

canonical link function $g(\xi) = -1/\xi$

Exponential family VI

Bernoulli distribution:

$$\begin{aligned}f(y; p) &= p^y(1-p)^{1-y} = \exp \left\{ y \log \frac{p}{1-p} + \log(1-p) \right\} \\&= \exp \{ y\theta - \log(1 + \exp(\theta)) \} \\&= \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\}\end{aligned}$$

with $\theta = \log \left(\frac{p}{1-p} \right)$, $a(\phi) = 1$, $b(\theta) = \log(1 + \exp(\theta))$, $c(y, \phi) = 0$

$$\mathbb{E}[Y] = b'(\theta) = \frac{\exp(\theta)}{1 + \exp(\theta)} = p,$$

$$\text{Var}[Y] = b''(\theta)a(\phi) = \frac{\exp(\theta)}{[1 + \exp(\theta)]^2} = p(1-p)$$

canonical link function $g(\xi) = \log \left(\frac{\xi}{1-\xi} \right)$, the logit function

Exponential family VII

Poisson distribution:

$$\begin{aligned}f(y; \lambda) &= \exp\{-\lambda\} \frac{\lambda^y}{y!} \\&= \exp\{y \log \lambda - \lambda - \log(y!)\} \\&= \exp\{y\theta - \exp(\theta) - \log(y!)\} \\&= \exp\left\{\frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi)\right\}\end{aligned}$$

with $\theta = \log(\lambda)$, $a(\phi) = 1$, $b(\theta) = \exp(\theta)$, and $c(y, \phi) = -\log(y!)$

$$\mathbb{E}[Y] = b'(\theta) = \exp(\theta) = \lambda,$$

$$\text{Var}[Y] = b''(\theta)a(\phi) = \exp(\theta) = \lambda$$

canonical link function $g(\xi) = \log(\xi)$, the logarithmic function

Estimation of a GLM I

- In the sequel we denote $Y \sim EF(\theta, \phi, a, b, c)$ if Y has a distribution that belongs to an exponential family
- With n observations, we write the model indexed with i to note that it is being applied to each observation.

$$Y_i \sim EF(\theta_i, \phi, a, b, c)$$

- We define a **link** between the mean and expected value of the distribution:

$$\eta_i = g(\mu_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)}$$

- Often we choose the **canonical link**: $\eta = \theta = g(\mu)$
- The parameters, β , of a GLM can be estimated using maximum likelihood.

Estimation of a GLM II

- The log-likelihood for a single observation

$$l(\theta_i, y_i) = \left\{ \frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right\}$$

where $\theta_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_{p-1} x_{i(p-1)}$.

- Then the log-likelihood is

$$\ell(\beta) = \sum_{i=1}^n \left\{ \frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right\}$$

- Unfortunately, unlike ordinary linear regression, there is no analytical solution for this maximization problem.
- Instead, it will need to be solved using numerical approximations via an iteratively reweighted least squares algorithm (IRLS).

Estimation of a GLM III

- To obtain the maximum likelihood estimator we need to find the root for the score:

$$\mathbf{s}(\beta) = \frac{\partial \ell(\beta)}{\partial \beta} = \sum_{i=1}^n \frac{dl_i}{d\theta_i} \frac{d\theta_i}{d\mu_i} \frac{\partial \mu_i}{\partial \beta} = \sum_{i=1}^n \frac{y_i - b'(\theta_i)}{a(\phi)} \frac{1}{V_i} \frac{\partial \mu_i}{\partial \beta}$$

where $\text{var}(Y_i|\beta) = a(\phi)V_i$ with

$$V_i = \frac{d^2 b(\theta)}{d\theta_i^2} = \frac{d^2 \mu_i}{d\theta_i^2}$$

- We re-write this in matrix form as

$$\mathbf{s}(\beta) = \sum_{i=1}^n \left(\frac{\partial \mu_i}{\partial \beta} \right)^\top \frac{Y_i - b'(\theta_i)}{a(\phi)V_i} = \mathbf{D}\mathbf{T}^{-1}(\mathbf{y} - \boldsymbol{\mu}(\beta))/\phi$$

Taking: $D_{ij} = \partial \mu_i / \partial \beta_j$ and $\mathbf{T} = \text{diag}(V_1, \dots, V_n)$.

Estimation of a GLM IV

- Notice that the expected information matrix for the model is:

$$I(\beta) = \mathbb{E} \left[\mathbf{S}(\beta) \mathbf{S}(\beta)^\top \right] = \mathbf{D}^\top \mathbf{T}^{-1} \mathbf{D} / a(\phi)$$

Since $\mathbf{D}$ and $\mathbf{T}$ both depend on β we will typically employ the observed information matrix evaluated at $\hat{\beta}$:

$$I(\hat{\beta}) = \hat{\mathbf{D}}^\top \hat{\mathbf{T}}^{-1} \hat{\mathbf{D}} / a(\phi)$$

Estimation of a GLM $\mathbf{V}$

- We now rewrite the score function for an individual β_j , remembering that $\eta_i = g(\mu_i)$:

$$S_j(\beta) = \frac{\partial \ell(\beta)}{\partial \beta_j} = \sum_{i=1}^n \frac{dl_i}{d\theta_i} \frac{d\theta_i}{d\mu_i} \frac{d\mu_i}{d\eta_i} \frac{\partial \eta_i}{\partial \beta_j} = \sum_{i=1}^n \frac{y_i - b'(\theta_i)}{a(\phi)V_i} \frac{\partial \mu_i}{\partial \eta_i} x_{ij}$$

therefore the (j, k) th element of the expected information is:

$$I_{jk}(\beta) = - \sum_{i=1}^n \mathbb{E} \left[\frac{\partial \ell(\beta)}{\partial \beta_j} \frac{\partial \ell(\beta)}{\partial \beta_k} \right] = \sum_{i=1}^n \frac{x_{ij} x_{ik}}{a(\phi)V_i} \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2$$

and the information matrix takes the form

$$\mathbf{I}(\beta) = \mathbf{X}^\top \mathbf{V}(\beta) \mathbf{X} \quad \text{where } \mathbf{V}(\beta) = \text{diag} \left(\frac{(\partial \mu_1 / \partial \eta_1)^2}{a(\phi)V_1}, \dots, \frac{(\partial \mu_n / \partial \eta_n)^2}{a(\phi)V_n} \right)$$

Estimation of a GLM VI

- We are trying to find β such that $S(\beta) = 0$. Use a Taylor expansion about $\beta^{(0)}$:

$$S(\beta) \approx S(\beta^{(0)}) + (\beta - \beta^{(0)})^\top S'(\beta^{(0)})$$

which leads to

$$\beta = \beta^{(0)} - (S'(\beta^{(0)}))^{-1} S(\beta^{(0)})$$

- The Newton-Rapson algorithm iterates the step

$$\beta^{(t+1)} = \beta^{(t)} - (S'(\beta^{(t)}))^{-1} S(\beta^{(t)})$$

for $t = 0, 1, \dots$ till convergence.

- Notice that $S'(\beta^{(0)})$ is the observed information matrix $I(\beta)$
- (If we substitute $I(\beta)$ with $I(\hat{\beta})$ the algorithm is called *Fisher scoring method*)
- Recall that we found that $I(\beta) = \mathbf{X}^\top \mathbf{V}(\beta) \mathbf{X}$.

Estimation of a GLM VII

- We also define a vector $\mathbf{u}^{(t)}$ and a vector $\mathbf{z}^{(t)}$ with elements

$$u_i^{(t)} = \frac{y_i - \mu_i^{(t)}}{a(\phi)V_i} \frac{d\mu_i}{d\eta_i} \bigg|_{\boldsymbol{\beta}^{(t)}} \quad \text{and} \quad z_i^{(t)} = \mathbf{X}\boldsymbol{\beta}^{(t)} + (y_i - \mu_i^{(t)}) \frac{d\eta_i}{d\mu_i} \bigg|_{\boldsymbol{\beta}^{(t)}}$$

Estimation of a GLM VIII

- We can rewrite

$$\begin{aligned}\beta^{(t+1)} &= \beta^{(t)} - (S'(\beta^{(t)}))^{-1} S(\beta^{(t)}) \\ &= \beta^{(t)} + (\mathbf{X}^\top \mathbf{V}(\beta^{(t)})^{-1} \mathbf{X}^\top \mathbf{u}^{(t)}) \\ &= (\mathbf{X}^\top \mathbf{V}(\beta^{(t)}) \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{V}(\beta^{(t)}) (\mathbf{X} \beta^{(t)} + \mathbf{V}(\beta^{(t)})^{-1} \mathbf{u}^{(t)}) \\ &= (\mathbf{X}^\top \mathbf{V}(\beta^{(t)}) \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{V}(\beta^{(t)}) \mathbf{z}^{(t)}\end{aligned}$$

- The Newton-Rapson method is equivalent to carrying out a weighted least square on the *working variable* $\mathbf{z}$
- The weight for this least square estimation are related to the variance of the distribution
- The method is therefore also known as the Iterative Reweighted Least Square (IRLS)
- Likelihoods from Exponential family tend to be well behaved: it is possible to maximise the likelihood

Estimation of a GLM IX

- The IRLS algorithm in summary:

- ➊ Initialize $\hat{\eta}^0$ (with some $\hat{\beta}^0$) and $\hat{\mu}^0$
- ➋ Create $z^0 = \hat{\eta}^0 + (y - \hat{\mu}^0) \frac{d\eta}{d\mu} |_{\hat{\eta}^0}$
- ➌ Form the weights

$$(v^0)^{-1} = \left(\frac{d\eta}{d\mu} \right)^2 |_{\hat{\eta}^0} V(y) |_{\hat{\mu}^0}$$

- ➍ Estimate $\hat{\beta}^1$ to get $\hat{\eta}^1$
 - ➎ Repeat steps 3-4-5 till convergence (on $\hat{\eta}$)
- Typically quite fast (even if sometimes it can fail)
 - No need to program it ourselves: in R we can use `glm`

R's glm function

- The `glm` function fit a GLM model using the maximum likelihood method
- For ordinary usage, the call is the same as for `lm`, except for one extra argument, `family`.
- The default value of `family` is `gaussian`
- In the Poisson regression case the call looks like
`glm(y~x, family = poisson)`
- In the logistic regression case, for example, the call looks like
`glm(y~x, family = binomial)`
- For each family we can specify a link function

GLM: the bernoulli Model I

- A survey investigating people's voting intention before the constitutional referendum in Chile in 1988.
- We predict the intention to vote "yes" as a function of other predictors (age, region, etc.):

```
logitout <- glm(yesVote ~ . , data = chileElection,  
               family = binomial)
```

```
summary(logitout)
```

Call:

```
glm(formula = yesVote ~ ., family = binomial, data = chileElection)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.046e+00	4.592e-01	2.279	0.02269 *
regionM	7.072e-01	6.023e-01	1.174	0.24030
regionN	-9.958e-02	3.587e-01	-0.278	0.78134
regionS	-3.044e-01	2.928e-01	-1.040	0.29847
regionSA	-3.012e-01	3.404e-01	-0.885	0.37619
population	1.276e-06	1.414e-06	0.902	0.36714
sexM	-5.515e-01	2.041e-01	-2.702	0.00689 **
age	7.108e-04	7.472e-03	0.095	0.92422
educationPS	-9.676e-01	3.461e-01	-2.795	0.00518 **

GLM: the bernoulli Model II

```
educationS -6.575e-01 2.440e-01 -2.695 0.00705 **
income      -2.972e-06 2.856e-06 -1.041 0.29807
statusquo   3.229e+00 1.524e-01 21.184 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2360.29  on 1702  degrees of freedom
Residual deviance:  703.48  on 1691  degrees of freedom
(54 observations deleted due to missingness)
AIC: 727.48

Number of Fisher Scoring iterations: 6
```

- `yesVote` is a binary variable. See `?binomial` to see how to handle binomial responses (e.g. number of exams passed out of k exams tried in an academic year).

GLM: the bernoulli Model III

- As a default R uses the canonical link:

```
binomial()$link
```

```
[1] "logit"
```

- The link can be changed - see ?family
- Specifying the family implies the specification of the variance function:

```
binomial()$variance
```

```
function (mu)
```

```
mu * (1 - mu)
```

```
<bytecode: 0x0000022a38f30ce0>
```

```
<environment: 0x0000022a3941eb70>
```

- Specifying the family and the link implies a number of relationships:

```
names(binomial())
```

```
[1] "family"      "link"         "linkfun"      "linkinv"      "variance"
```

```
[6] "dev.resids"  "aic"          "mu.eta"       "initialize"    "validmu"
```

```
[11] "valideta"    "simulate"     "dispersion"
```

Fitting issues for the logistic regression I

- We should note that, if there exists some β^* such that

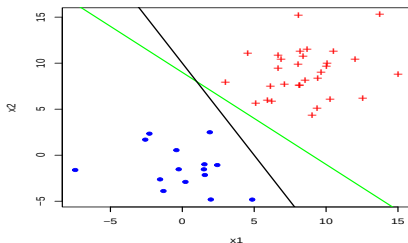
$$\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)} > 0 \implies y_i = 1$$

and

$$\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)} < 0 \implies y_i = 0$$

for all observations, then the MLE is not unique.

Fitting issues for the logistic regression II



- Such data is said to be separable.
- This, and similar numeric issues related to estimated probabilities near 0 or 1, will return a warning in R.

Fitting issues for the logistisc regression III

Warning messages:

1: glm.fit: algorithm did not converge

2: glm.fit: fitted probabilities numerically 0 or 1 occurred

- When this happens, the model is still “fit”, but there are consequences, namely, the estimated coefficients are highly suspect.
- This is an issue when then trying to interpret the model.
- However it could be useful for creating a classifier

Inference for model parameters I

- R prints a summary with some information on the regression parameter estimates
- The assumptions on which a generalized linear model is constructed allow us to specify what is the asymptotic distribution of the random vector $\hat{\beta}$ through the theory of MLE
- We assume that the randomness of Y comes only from $(Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1})$ and not from the predictors.
- We rewrite the relationship between the expected value of μ and the linear predictor in matrix form:

$$\eta = g(\mu) = \mathbf{X}\beta$$

- β is a vector of parameters that we wish to estimate: we estimate them using an algorithm that maximizes the likelihood
- so the β parameters enjoy optimal (approximate) properties of MLEs

Inference for model parameters II

There is an important difference between the inference results for the Gaussian linear model and for glm:

- In Gaussian linear model the inference is exact. This is due to the nice properties of the normal, least squares estimation, and linearity. As a consequence, the distributions of the coefficients are perfectly known assuming that the assumptions hold.
- In generalized linear models the inference is asymptotic. This means that the distributions of the coefficients are unknown except for large sample sizes n , for which we have approximations. Keep in mind that in GLM-like models the relationship between $\mu(x)$ and x is non-linear: we fit a more complex model and this hinders our ability to make inference.

Inference for model parameters III

- We can show that

$$\hat{\beta} \stackrel{\text{approx}}{\sim} \mathcal{N}(\beta, \mathcal{I}(\beta)^{-1})$$

where $\stackrel{\text{approx}}{\sim}$ must be understood as asymptotically distributed as when $n \rightarrow \infty$ and

$$\mathcal{I}(\beta) = \mathbb{E} \left[-\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^\top} \right]$$

is the Fisher information matrix.

- The "larger" (large eigenvalues) the matrix is, the more precise the estimation of β is, because that results in smaller variances.

Inference for model parameters IV

- As seen before, it turns out that

$$\mathcal{I}(\beta) = \mathbf{X}^\top \mathbf{V}(\beta) \mathbf{X}$$

where

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1(p-1)} \\ 1 & x_{21} & x_{22} & \cdots & x_{2(p-1)} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n(p-1)} \end{bmatrix}$$

and $\mathbf{V}(\beta) = \text{diag}(V_1, \dots, V_n)$ with $V_i = \frac{1}{\text{Var}[Y_i]} \left(\frac{d\mu}{d\eta} \right)^2$.

- The $\mathbf{V}(\beta)$ matrix is used as a weight matrix in the IRLS
- Notice that in the gaussian linear regression (with identity link) V_i are a constant for all i

Inference for model parameters V

- V_i values for noticeable distributions:
 - for logistic regression

$$V_i = \frac{\exp(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)})}{[1 + \exp(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)})]^2}$$

- for Poisson regression

$$V_i = \exp\{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{i(p-1)}\}$$

- We do not know the true value of $\beta \dots$
- The inverse of the Fisher information matrix is estimated by plugging in $\hat{\beta}$ into $\mathcal{I}(\beta)^{-1}$, i.e. by $\mathcal{I}(\hat{\beta})^{-1}$

Inference for model parameters VI

Quality of the estimation:

- Bias. The estimates are **asymptotically** unbiased.
- Variance depends on:
 - Sample size n . Hidden inside $\mathbf{X}^\top \mathbf{V}(\beta)^{-1} \mathbf{X}$. As n grows, the precision of the estimators increases.
 - Weighted predictor sparsity $(\mathbf{X}^\top \mathbf{V}(\beta)^{-1} \mathbf{X})^{-1}$. The more sparse the predictor is (small eigenvalues of $(\mathbf{X}^\top \mathbf{V}(\beta)^{-1} \mathbf{X})^{-1}$), the more precise $\hat{\beta}$ is.
- The precision of $\hat{\beta}$ is affected by the value of β , which is “hidden” inside $\mathbf{V}(\beta)$
- This contrasts sharply with the linear model, where the precision of the least squares estimator was not affected by the value of the unknown coefficients.
- This is partially due to the heteroskedasticity of logistic regression and Poisson regression, which implies a dependence of the variance of Y in the predictors, hence in β .

Confidence intervals for the coefficients

- Similar to linear regression, the problem is that $\mathbf{V}(\beta)$ is unknown in practice because it depends on β .
- Plugging-in the estimate $\hat{\beta}$ to β in $\mathbf{V}(\beta)$ results in $\hat{\mathbf{V}}$.
- We can use $\hat{\mathbf{V}}$ to get

$$Z_j = \frac{\hat{\beta}_j - \beta_j}{\text{SE}(\hat{\beta}_j)} \underset{\text{approx}}{\sim} \mathcal{N}(0, 1)$$

where $\text{SE}(\hat{\beta}_j)^2 = v_j$ with v_j is the j -th element of the diagonal of $(\mathbf{X}^\top \hat{\mathbf{V}}^{-1} \mathbf{X})^{-1}$

- Thanks to normal approximation, we can have the $100(1 - \alpha)\%$ CI for the coefficient β_j

$$\hat{\beta}_j \pm z_{1-\alpha/2} \text{SE}(\hat{\beta}_j)$$

where $z_{1-\alpha/2}$ is the $1 - \alpha/2$ -upper quantile of the $\mathcal{N}(0, 1)$.

Testing with GLMs: Wald test I

- System of hypothesis

$$H_0 : \beta_j = 0 \quad \text{vs} \quad H_A : \beta_j \neq 0$$

- The distribution of the test statistic

$$W = \frac{\hat{\beta}_j}{\text{SE}[\hat{\beta}_j]}$$

is no longer T , but for a large number of observations ($n \rightarrow \infty$)

$$W \overset{\text{approx}}{\sim} \mathcal{N}(0, 1)$$

Testing with GLMs: Wald test II

- The t -test for ordinary linear regression, assuming the assumptions were correct, had an exact distribution for any sample size.
- R will obtain the standard error for us. The use of this test will be extremely similar to the t -test for ordinary linear regression. Essentially the only thing that changes is the distribution of the test statistic.

GLM: the Poisson Model I

- This problem refers to data from a study of nesting horseshoe crabs (J. Brockmann, Ethology 1996);
- Available from the GLMsData package
- Each female horseshoe crab in the study had a male crab attached to her in her nest. The study investigated factors that affect whether the female crab had any other males, called satellites, residing near her.

GLM: the Poisson Model II



- Explanatory variables that are thought to affect this included the female crab's color (C), spine condition (S), weight (Wt), and carapace width (W). The response outcome for each female crab is her number of satellites (Sa).
- There are 173 females in this study and the data can be derived from the library GLMsData

GLM: the Poisson Model III

```
# install.packages("GLMsData")
data(hcrabs, package = "GLMsData")
### here do some exploration of the data
```

- We fit the intercept only model. This model implies the expected number of satellites per each crab is the same

```
model0<-glm(Sat~1, family=poisson(link=log),data=hcrabs)
summary(model0)
```

Call:

```
glm(formula = Sat ~ 1, family = poisson(link = log), data = hcrabs)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.0713	0.0445	24.07	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

GLM: the Poisson Model IV

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 632.79 on 172 degrees of freedom
Residual deviance: 632.79 on 172 degrees of freedom
AIC: 990.09

Number of Fisher Scoring iterations: 5

GLM: the Poisson Model V

- Then we fit a model including the carapace width and weight as predictors

```
model1<-glm(Sat~1+Width+Wt,family=poisson(link=log),  
            data=hcrabs)  
summary(model1)
```

Call:

```
glm(formula = Sat ~ 1 + Width + Wt, family = poisson(link = log),  
     data = hcrabs)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.2952111	0.8988960	-1.441	0.14962
Width	0.0460765	0.0467497	0.986	0.32433
Wt	0.0004470	0.0001586	2.818	0.00483 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

GLM: the Poisson Model VI

```
Null deviance: 632.79  on 172  degrees of freedom  
Residual deviance: 559.90  on 170  degrees of freedom  
AIC: 921.2
```

```
Number of Fisher Scoring iterations: 6
```

- Natural question: does adding the predictors "improve" the model?
- Notice that we have fitted two nested models

Testing with GLMs: likelihood-ratio test I

- Consider the following full model,

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

and we denote the MLE of these β -parameters as $\hat{\beta}_{\text{Full}}$

- Consider a null model,

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{q-1} x_{q-1}$$

where $q < p$. We denote the MLE of these β -parameters as $\hat{\beta}_{\text{Null}}$

- The difference between these two models can be codified by the null hypothesis of a test:

$$H_0 : \beta_q = \beta_{q+1} = \dots = \beta_{p-1} = 0 \quad \text{VS} \quad H_A : \text{any of } \beta_q, \beta_{q+1}, \dots, \beta_{p-1} \neq 0$$

Testing with GLMs: likelihood-ratio test II

- We define a test statistic, LR ,

$$LR = -2 \log \left(\frac{L(\hat{\beta}_{\text{Null}})}{L(\hat{\beta}_{\text{Full}})} \right) = 2 \log \left(\frac{L(\hat{\beta}_{\text{Full}})}{L(\hat{\beta}_{\text{Null}})} \right) = 2 \left(\ell(\hat{\beta}_{\text{Full}}) - \ell(\hat{\beta}_{\text{Null}}) \right)$$

where L denotes a likelihood and ℓ denotes a log-likelihood.

- For a large enough sample, this test statistic has an approximate Chi-square distribution

$$LR \overset{\text{approx}}{\sim} \chi^2_{p-q}$$

- The test, which we will call the **Likelihood-Ratio Test (LRT)**, will be the analogue to the ANOVA F -test for linear regression.
- To perform the LRT, we'll actually again use the `anova` function in R.
- The LRT is a rather general test, however, here we have presented a specific application to GLMs.

Testing with GLMs: likelihood-ratio test III

- We use the LRT test to compare the two models for the horseshoe crabs

```
logLik(model0); logLik(model1)
'log Lik.' -494.0447 (df=1)
'log Lik.' -457.5991 (df=3)
(tstat <- as.numeric(2*(logLik(model1) - logLik(model0))))
[1] 72.89106
diff_df <- length(model1$coefficients) - length(model0$coefficients)
# pvalue
pchisq(tstat, df = diff_df, lower.tail = FALSE)
[1] 1.485621e-16
```

- Or we can use directly the anova function in R:

```
anova(model0, model1, test = "LRT")
```


Testing with GLMs: likelihood-ratio test IV

Analysis of Deviance Table

Model 1: $\text{Sat} \sim 1$

Model 2: $\text{Sat} \sim 1 + \text{Width} + \text{Wt}$

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	172	632.79			
2	170	559.90	2	72.891	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

- Notice that we need to specify the test we wish to perform
- The table produce by anova is referred to as the *analysis of deviance* table
- The *deviance* plays in GLM the same role played by the sum of squares in gaussian linear regression

Deviance I

- The deviance for a model is defined as:

$$\begin{aligned} D &= 2[l(\hat{\beta}_{\max}) - l(\hat{\beta})]\phi \\ &= \frac{\phi}{a(\phi)}[y(\hat{\theta}^{\text{sat}} - \hat{\theta}) - b(\hat{\theta}^{\text{sat}}) + b(\hat{\theta})] \end{aligned}$$

where $l(\hat{\beta}_{\max})$ is the maximized likelihood under the saturated model and $\hat{\theta}^{\text{sat}}$ is the estimated value of θ in the saturated model

- The saturated model is the model in which we have a different estimated value θ_i for each observation
- The likelihood under the saturated model is the highest possible likelihood for the data

Deviance II

Saturated model

- $g(\mathbb{E}[Y_i | X_1 = x_{i1}, \dots, X_{p-1} = x_{i,p-1}]) = \theta_i, (i = 1, \dots, n)$
- The log-likelihood for a single observation is

$$\log f(y_i; \theta_i, \phi) = \frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi)$$

- Taking the derivative with respect to θ_i

$$\frac{\partial}{\partial \theta_i} \log f(y_i; \theta_i, \phi) = \frac{y_i - b'(\theta_i)}{a(\phi)} = \frac{y_i - \mathbb{E}[Y_i | X_1 = x_{i1}, \dots, X_{p-1} = x_{i,p-1}]}{a(\phi)}$$

- MLE $\frac{\partial}{\partial \theta_i} \log f(y_i; \theta_i, \phi) = 0, \Rightarrow y_i = b'(\hat{\theta}_i), \Rightarrow \hat{\theta}_i^{\text{sat}} = b'^{-1}(y_i)$ and $\hat{y}_i^{\text{sat}} = y_i$

Deviance III

Residual deviance

Residual deviance D is 2 times the log of the ratio between the likelihood L^{sat} for the saturated model and the likelihood $L(\hat{\beta}_0, \dots, \hat{\beta}_{p-1})$ for the fitted model.

$$D = 2 \log \left(\frac{L^{\text{sat}}}{L(\hat{\beta})} \right) = 2 \left(\ell^{\text{sat}} - \ell(\hat{\beta}) \right)$$

For independent observations we have

$$D = 2 \sum_{i=1}^n \left(\ell_i^{\text{sat}} - \ell_i(\hat{\beta}) \right) = 2 \sum_{i=1}^n d_i$$

d_i is the individual deviance.

Deviance IV

Poisson regression example:

- Remember that the log-likelihood, ℓ_i for a single observation is

$$\ell_i = y_i \log \mu_i - \mu_i \quad \text{where } \mathbb{E}[Y_i] = \mu_i$$

- We have

$$D = 2 \left(\ell^{\text{sat}} - \ell(\hat{\beta}) \right) = 2 \sum_{i=1}^n [(y_i \log \hat{\mu}_i^{\text{sat}} - \hat{\mu}_i^{\text{sat}}) - (y_i \log \hat{\mu}_i - \hat{\mu}_i)]$$

where $\hat{\mu}_i^{\text{sat}} = y_i$, $\hat{\mu}_i = \exp\{\hat{\beta}_0 + \hat{\beta}_1 x_{i,1} + \cdots + \hat{\beta}_{p-1} x_{i,p-1}\}$

Then:

$$D = 2 \sum_{i=1}^n [(y_i \log y_i - y_i) - (y_i \log \hat{\mu}_i - \hat{\mu}_i)]$$

where $y_i \log y_i = 0$ for $y_i = 0$

Deviance V

For large n

$$D \stackrel{\text{approx}}{\sim} \chi_{n-p}^2$$

where p is the number of parameters for current model.

D may be used for goodness-of-fit assessment.

Null deviance

Null deviance is 2 times the log of the ratio between the likelihood L^{sat} for the saturated model and the likelihood $L(\hat{\beta}_0)$ for the fitted model with only intercept.

Deviance VI

In R we can access the deviance in the summary of through the deviance function:

```
summary(model1)$deviance; summary(model1)$null.dev
```

```
[1] 559.9006
```

```
[1] 632.7917
```

```
deviance(model1); deviance(model0)
```

```
[1] 559.9006
```

```
[1] 632.7917
```

Deviance VII

We can also define the scale deviance which does not depend on ϕ :

$$D^* = D/\phi$$

and, in the case in which ϕ is known, the LRT can be rewritten as the difference in deviances between two models

$$\begin{aligned} D(\mathcal{M}_1) - D(\mathcal{M}_2) &= 2[(l(\hat{\beta}_{\max}) - l(\hat{\beta}^{\mathcal{M}_1})) - (l(\hat{\beta}_{\max}) - l(\hat{\beta}^{\mathcal{M}_2}))] \\ &= 2[l(\hat{\beta}^{\mathcal{M}_2}) - l(\hat{\beta}^{\mathcal{M}_1})] \end{aligned}$$

This is no longer true when ϕ needs to be estimated (for example, in a gaussian or gamma distribution)

Goodness of fit I

- LRT only holds for nested models
- How to compare non-nested models?
- We need general purpose measures of goodness of fit
- We use AIC (and BIC):

$$AIC(\theta(\mathcal{M}), p(\mathcal{M})) = -\log\text{Lik}(\theta(\mathcal{M})) + 2 * p(\mathcal{M})$$

```
AIC(model1)
```

```
[1] 921.1983
```

```
2*(-as.numeric(logLik(model1)) +  
  length(model1$coefficients))
```

```
[1] 921.1983
```

- Same interpretation as for linear models

SAheart example I

- To illustrate the use of logistic regression, we will use the SAheart dataset from the book *The Elements of Statistical Learning*.

```
fl <- "http://www-stat.stanford.edu/~tibs/ElemStatLearn/datasets/SAheart.  
SAheart <- read.table(fl,  
                      sep=" ", head=T, row.names=1)
```

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age	chd
1	160	12.00	5.73	23.11	Present	49	25.30	97.20	52	1
2	144	0.01	4.41	28.61	Absent	55	28.87	2.06	63	1
3	118	0.08	3.48	32.28	Present	52	29.14	3.81	46	0
4	170	7.50	6.41	38.03	Present	51	31.99	24.26	58	1
5	134	13.60	3.50	27.78	Present	60	25.99	57.34	49	1

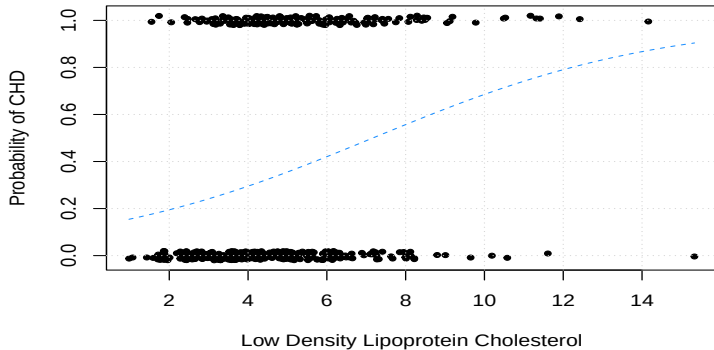
- This data comes from a retrospective sample of males in a heart-disease high-risk region of the Western Cape, South Africa.

SAheart example II

- The response variable is `chd`, which indicates whether or not coronary heart disease is present in an individual. (numeric 0 / 1) variable.
- Predictors: various measurements for each individual, many related to heart health. For example `sbp`, systolic blood pressure, and `ldl`, low density lipoprotein cholesterol.
- First we model the probability of coronary heart disease based on low density lipoprotein cholesterol:

$$\log \left(\frac{\Pr[\text{chd} = 1]}{1 - \Pr[\text{chd} = 1]} \right) = \beta_0 + \beta_{\text{ldl}} x_{\text{ldl}}$$

SAheart example III



As we would expect as ldl increases, so does the probability of chd.

SAheart example IV

We wish to carry out a test for $H_0 : \beta_{\text{ldl}} = 0$ VS $H_A : \beta_{\text{ldl}} \neq 0$

We find the test statistic and p-value for the test using the summary function

```
coef(summary(chd_mod_ldl))
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.9686681	0.27307908	-7.209150	5.630207e-13
ldl	0.2746613	0.05163983	5.318787	1.044615e-07

- We have a low p-value and reject H_0 : ldl appears to be a relevant predictor.
- Confidence intervals at 95%

```
confint.default(chd_mod_ldl, level=0.95)
```

	2.5 %	97.5 %
(Intercept)	-2.5038933	-1.4334430
ldl	0.1734491	0.3758735

SAheart example V

- To fit an additive model using all available predictors, we use:

```
chd_mod_additive <- glm(chd ~ ., data = SAheart,  
                        family = binomial)
```

- We use the likelihood-ratio test to compare the two model. Specifically, we are testing

$$H_0 : \beta_{\text{sbp}} = \beta_{\text{tobacco}} = \beta_{\text{adiposity}} = \beta_{\text{famhist}} = \beta_{\text{typea}} = \beta_{\text{obesity}} = \beta_{\text{alcohol}} = \beta_{\text{age}} = 0$$

- The LR test statistic,

```
-2 * as.numeric(logLik(chd_mod_ldl) - logLik(chd_mod_additive))  
[1] 92.13879
```

SAheart example VI

- We can also use the `anova` function, by specifying `test = "LRT"`

```
anova(chd_mod_ldl, chd_mod_additive, test = "LRT")
```

Analysis of Deviance Table

Model 1: `chd ~ ldl`

Model 2: `chd ~ sbp + tobacco + ldl + adiposity + famhist + typea + obesity +
alcohol + age`

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	460	564.28			
2	452	472.14	8	92.139	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The small p-value suggests that we prefer the larger model.

- To select a subset of predictors, we can use a stepwise procedure as we did with ordinary linear regression.
- Recall that AIC and BIC were defined in terms of likelihoods.

SAheart example VII

- Example using AIC with a backwards selection procedure.

```
chd_mod_selected <- step(chd_mod_additive, trace = 1, k = 2)
```

Start: AIC=492.14

```
chd ~ sbp + tobacco + ldl + adiposity + famhist + typea + obesity +  
      alcohol + age
```

	Df	Deviance	AIC
- alcohol	1	472.14	490.14
- adiposity	1	472.55	490.55
- sbp	1	473.44	491.44
<none>		472.14	492.14
- obesity	1	474.23	492.23
- ldl	1	481.07	499.07
- tobacco	1	481.67	499.67
- typea	1	483.05	501.05
- age	1	486.53	504.53
- famhist	1	488.89	506.89

Step: AIC=490.14

SAheart example VIII

```
chd ~ sbp + tobacco + ldl + adiposity + famhist + typea + obesity +  
age
```

	Df	Deviance	AIC
- adiposity	1	472.55	488.55
- sbp	1	473.47	489.47
<none>		472.14	490.14
- obesity	1	474.24	490.24
- ldl	1	481.15	497.15
- tobacco	1	482.06	498.06
- typea	1	483.06	499.06
- age	1	486.64	502.64
- famhist	1	488.99	504.99

Step: AIC=488.55

```
chd ~ sbp + tobacco + ldl + famhist + typea + obesity + age
```

	Df	Deviance	AIC
- sbp	1	473.98	487.98

SAheart example IX

```
<none>          472.55 488.55
- obesity  1    474.65 488.65
- tobacco  1    482.54 496.54
- ldl      1    482.95 496.95
- typea    1    483.19 497.19
- famhist  1    489.38 503.38
- age      1    495.48 509.48
```

Step: AIC=487.98

```
chd ~ tobacco + ldl + famhist + typea + obesity + age
```

```
      Df Deviance    AIC
- obesity  1    475.69 487.69
<none>          473.98 487.98
- tobacco  1    484.18 496.18
- typea    1    484.30 496.30
- ldl      1    484.53 496.53
- famhist  1    490.58 502.58
- age      1    502.11 514.11
```

SAheart example X

Step: AIC=487.69

```
chd ~ tobacco + ldl + famhist + typea + age
```

	Df	Deviance	AIC
<none>		475.69	487.69
- ldl	1	484.71	494.71
- typea	1	485.44	495.44
- tobacco	1	486.03	496.03
- famhist	1	492.09	502.09
- age	1	502.38	512.38

```
coef(chd_mod_selected)
```

(Intercept)	tobacco	ldl	famhistPresent	typea
-6.44644451	0.08037533	0.16199164	0.90817526	0.03711521
age				
0.05046038				

SAheart example XI

- We could again compare this model to the additive models.

$$H_0 : \beta_{\text{sbp}} = \beta_{\text{adiposity}} = \beta_{\text{obesity}} = \beta_{\text{alcohol}} = 0$$

```
anova(chd_mod_selected, chd_mod_additive, test = "LRT")
```

Analysis of Deviance Table

Model 1: `chd ~ tobacco + ldl + famhist + typea + age`

Model 2: `chd ~ sbp + tobacco + ldl + adiposity + famhist + typea + obesity +
alcohol + age`

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	456	475.69			
2	452	472.14	4	3.5455	0.471

- Here it seems that we would prefer the selected model with 5 predictors.

Prediction I

- Recall that

$$g(\mathbb{E}[Y|X_1 = x_1, \dots, X_{p-1} = x_{p-1}]) = \beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1}$$

- Predicted observation for the current model:

$$\hat{Y} = g^{-1}(\hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_{p-1} x_{p-1})$$

- Logistic regression

$$\hat{Y} = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_{p-1} x_{p-1}}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_{p-1} x_{p-1}}}$$

- Poisson regression

$$\hat{y} = \exp\{\hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_{p-1} x_{p-1}\}$$

Prediction II

- In R we can obtain a prediction for both $\hat{\eta}$ and $\hat{\mu} = g^{-1}(\hat{\eta})$:

```
## back to the hcrabs example
```

```
head(predict(model1, type = "link"),4)
```

1	2	3	4
1.3720122	0.4343135	0.9308087	0.7861229

```
exp(head(predict(model1, type = "link"),4))
```

1	2	3	4
3.943277	1.543903	2.536560	2.194870

```
head(predict(model1, type = "response"),4)
```

1	2	3	4
3.943277	1.543903	2.536560	2.194870

Prediction III

- Since $\hat{\beta} \stackrel{\text{approx}}{\sim} \mathcal{N}(\beta, \mathcal{I}(\beta)^{-1})$ we can construct (approximate) confidence intervals for η .
- Since $\hat{\eta} = X\hat{\beta}$:

$$\hat{V}(\hat{\eta}) = X\hat{V}(\hat{\beta})X^{\top}$$

- Confidence interval for η_i (confidence level: $1-\alpha$):

$$\left(\hat{\eta}_i + z_{\alpha/2} \left(\hat{V}(\hat{\eta}) \right)_{ii}, \hat{\eta}_i + z_{1-\alpha/2} \left(\hat{V}(\hat{\eta}) \right)_{ii} \right)$$

- We can then back-transform the lower and upper bound of the confidence interval for the linear predictor to obtain (approximate) confidence intervals for μ_i , since $\hat{\mu}_i = g^{-1}(\hat{\eta}_i)$:

$$\left(g^{-1} \left(\hat{\eta}_i + z_{\alpha/2} \left(\hat{V}(\hat{\eta}) \right)_{ii} \right), g^{-1} \left(\hat{\eta}_i + z_{1-\alpha/2} \left(\hat{V}(\hat{\eta}) \right)_{ii} \right) \right)$$

Prediction IV

- In R:

```
lpred <- predict(model1, type = "link", se.fit=TRUE)
cbind(lpred$fit[1:4] + qnorm(.025)*lpred$se.fit[1:4],
      lpred$fit[1:4] + qnorm(.975)*lpred$se.fit[1:4])
      [,1]      [,2]
1 1.2716445 1.4723799
2 0.2372926 0.6313344
3 0.8315315 1.0300860
4 0.6628823 0.9093635

exp(cbind(lpred$fit[1:4] + qnorm(.025)*lpred$se.fit[1:4],
          lpred$fit[1:4] + qnorm(.975)*lpred$se.fit[1:4]))
      [,1]      [,2]
1 3.566713 4.359598
2 1.267812 1.880118
3 2.296834 2.801307
4 1.940377 2.482742
```


GLM residuals I

- Residuals represent the difference between the data and the model and are essential to explore the adequacy of the model.
- In the Gaussian case, the residuals are $r_i = y_i - \hat{y}_i$.
- These are called **response residuals** for GLMs, but since the variance of the response is not constant for most GLMs, some modification is necessary.
- The **Pearson residuals** are comparable to the standardized residuals used in linear models and is defined as:

$$r_i^P = \frac{y_i - \hat{y}_i}{\sqrt{\hat{V}_i}}$$

GLM residuals II

- The deviance residuals r_i^D

$$D = \sum_{i=1}^n (r_i^D)^2 = \sum_{i=1}^n d_i$$

and

$$r_i^D = \text{sign}(y_i - \hat{y}_i) \sqrt{d_i}$$

- For example in the Poisson regression:

$$r_i^D = \text{sign}(y_i - \hat{y}_i) \sqrt{2(y_i \log y_i / \hat{y}_i - y_i + \hat{y}_i)}$$

- (notice that if $y_i = 0$ $y_i \log(y_i / \hat{y}_i)$ is replaced by 0, i.e. its value in the limit for $y \rightarrow 0$)

GLM residuals III

- (Crab data) We can obtain the deviance residuals as:

```
residuals(model1)[1:5]
```

1	2	3	4	5
1.7903699	-1.7572153	3.1414336	-2.0951707	0.6101256

These are the default choice of residuals.

- The Pearson residuals are:

```
residuals(model1, "pearson")[1:5]
```

1	2	3	4	5
2.0428979	-1.2425388	4.0582724	-1.4815094	0.6455594

GLM residuals IV

- If we use

```
model1$residuals[1:5]
```

1	2	3	4	5
1.0287693	-1.0000000	2.5481128	-1.0000000	0.3790497

we obtain the working residuals, i.e.

$$\hat{\eta}_i + (y_i - \hat{\mu}_i) \frac{d\eta_i}{d\mu_i}$$

- For linear models, the plot of residuals against fitted values is probably the single most valuable graphic for model diagnostics.
- For GLMs: which residuals? Which scale (linear predictor or original scale?)
- In GLMs residual plots are much harder to read and verifying assumptions is more complicated (so we don't really focus on it during the course)

GLM residuals V

- For GLMs, we must decide on the appropriate scale for the fitted values. Usually, it is better to plot the fitted linear predictors.

$$\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_{p-1} x_{i,p-1}$$

rather than the predicted responses $\hat{y}_i$.

- Scatter plot of Deviance residuals against x_j used to check whether any systematic relationship is present: if so include x_j in the model.
- Plot working residuals against linear predictor. Plot should be linear - if not the link function might be not correctly specified.
- Influential and leverage points can still be problematic: we can define leverage-aware residuals, but we don't discuss them in the course

Example: Galapagos data I

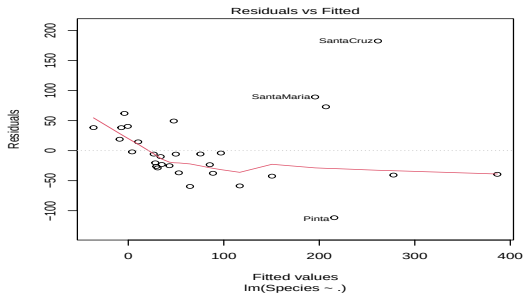
- For 30 Galápagos Islands, we have a count of the number of plant species found on each island and the number that are endemic to that island. We also have five geographic variables for each island.
- We model the number of species using normal linear regression:

```
data(gala, package="faraway")  
gala <- gala[,-2]
```

- We throw out the Endemics variable (which falls in the second column of the dataframe) since we won't be using it in this analysis. We fit a linear regression and look at the residual vs. fitted plot:

```
mod1 <- lm(Species ~ . , gala)  
plot(mod1, 1)
```

Example: Galapagos data II

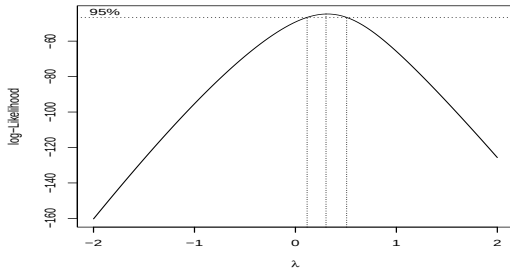


We see clear evidence of nonconstant variance

- the Box-Cox method reveals that a square-root transformation is a sensible transformation

```
library(MASS)
boxcox(mod1, plotit = TRUE)
```

Example: Galapagos data III



```
modt <- lm(sqrt(Species) ~ . , gala)
summary(modt)
```


Example: Galapagos data IV

Call:

```
lm(formula = sqrt(Species) ~ ., data = gala)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5572	-1.4969	-0.3031	1.3527	5.2110

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	3.3919243	0.8712678	3.893	0.000690	***
Area	-0.0019718	0.0010199	-1.933	0.065080	.
Elevation	0.0164784	0.0024410	6.751	5.55e-07	***
Nearest	0.0249326	0.0479495	0.520	0.607844	
Scruz	-0.0134826	0.0097980	-1.376	0.181509	
Adjacent	-0.0033669	0.0008051	-4.182	0.000333	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

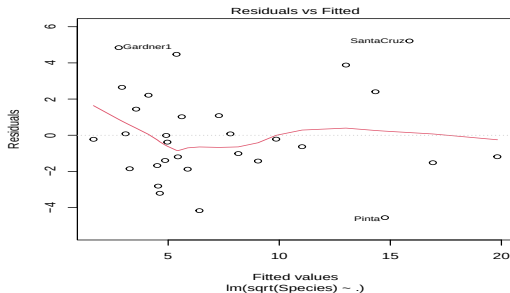
Residual standard error: 2.774 on 24 degrees of freedom

Example: Galapagos data V

Multiple R-squared: 0.7827, Adjusted R-squared: 0.7374

F-statistic: 17.29 on 5 and 24 DF, p-value: 2.874e-07

```
plot(modt, 1)
```



- We achieved this fit at the cost of transforming the response. This makes interpretation more difficult.

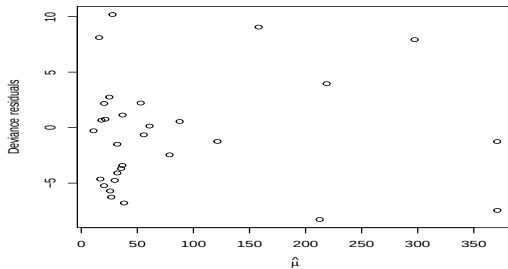
Example: Galapagos data VI

- Furthermore, some of the response values are quite small (single digits) which makes us question the validity of the normal approximation.
- This model may be adequate, but perhaps we can do better.

Example: Galapagos data VII

- Poisson regression

```
modp <- glm(Species ~ ., family=poisson, gala)
plot(residuals(modp) ~ predict(modp, type="response"),
     xlab=expression(hat(mu)),
     ylab="Deviance residuals")
```

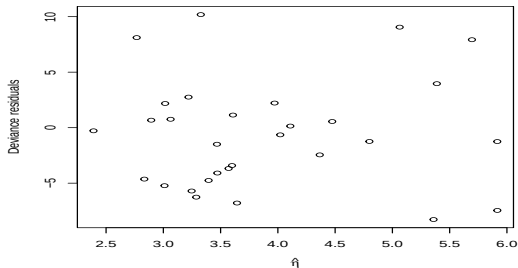


Example: Galapagos data VIII

- There are just a few islands with a large predicted number of species while most predicted response values are small. This makes it difficult to see the relationship between the residuals and the fitted values because most of the points are compressed on the left of the display.

```
plot(residuals(modp) ~ predict(modp,type="link"),  
      xlab=expression(hat(eta)),ylab="Deviance residuals")
```

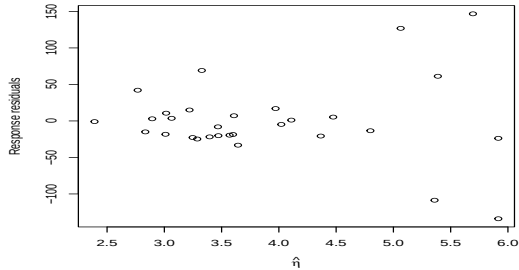
Example: Galapagos data IX



- If we use response residuals

```
plot(residuals(modp,type="response") ~ predict(modp,type="link"),  
     xlab=expression(hat(eta)),ylab="Response residuals")
```

Example: Galapagos data X



We see a pattern of increasing variation consistent with the Poisson.

Classification ¹

¹Material in these slides was heavily influenced by Chapter 17 of David Dal Piaz's [book](#).

Classification I

- Making predictions in the context of the logistic regression.
- Based on the values of the predictors, an observation is classified as $Y = 1$ or as $Y = 0$.
- Suppose that we know

$$p(x_1, \dots, x_{p-1}) = \Pr[Y = 1 \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}]$$

and

$$1 - p(x_1, \dots, x_{p-1}) = \Pr[Y = 0 \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}].$$

- Rule: we classify an observation to the class (0 or 1) with the larger probability. In general, this result is called

the Bayes Classifier

$$C^B(x_1, \dots, x_{p-1}) = \underset{k}{\operatorname{argmax}} P[Y = k \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}].$$

Classification II

- For a binary response, that is,

$$C^B(x_1, \dots, x_{p-1}) = \begin{cases} 1 & p(x_1, \dots, x_{p-1}) > 0.5 \\ 0 & p(x_1, \dots, x_{p-1}) \leq 0.5 \end{cases}$$

- Simply put, the Bayes classifier (not to be confused with the Naive Bayes Classifier) minimizes the probability of misclassification by classifying each observation to the class with the highest probability.
- Unfortunately, in practice, we won't know the necessary probabilities to directly use the Bayes classifier. Instead we'll have to use estimated probabilities.

$$\hat{C}^B(x_1, \dots, x_{p-1}) = \underset{k}{\operatorname{argmax}} \widehat{\Pr}[Y = k \mid X_1 = x_1, \dots, X_{p-1} = x_{p-1}].$$

Classification III

- In the case of a binary response since

$$\hat{p}(x_1, \dots, x_{p-1}) = 1 - \hat{p}(x_1, \dots, x_{p-1})$$

this becomes

$$\hat{C}(x_1, \dots, x_{p-1}) = \begin{cases} 1 & \hat{p}(x_1, \dots, x_{p-1}) > 0.5 \\ 0 & \hat{p}(x_1, \dots, x_{p-1}) \leq 0.5 \end{cases}$$

- To use logistic regression for classification, we first use logistic regression to obtain estimated probabilities, $\hat{p}(x_1, \dots, x_{p-1})$, then use these in conjunction with the above classification rule.
- Logistic regression is just one of many ways that these probabilities could be estimated.

spam Example I

- The spam dataset

```
spam <- read.table("https://web.stanford.edu/~hastie/ElemStatLearn/datasets/spam.data",  
                  header=FALSE)
```

```
names(spam) <- c("make", "address", "all", "num3d", "our", "over", "remove",  
                "internet", "order", "mail", "receive", "will", "people", "report",  
                "addresses", "free", "business", "email", "you", "credit", "your",  
                "font", "num000", "money", "hp", "hpl", "george", "num650", "lab",  
                "labs", "telnet", "num857", "data", "num415", "num85", "technology",  
                "num1999", "parts", "pm", "direct", "cs", "meeting", "original",  
                "project", "re", "edu", "table", "conference", "charSemicolon",  
                "charRoundbracket", "charSquarebracket", "charExclamation", "charDollar",  
                "charHash", "capitalAve", "capitalLong", "capitalTotal", "type")
```

- Created in the late 1990s at Hewlett-Packard Labs, it contains 4601 emails, of which 1813 are considered spam. The remaining are not spam.

spam Example II

- The response variable, `type`, is a **factor** with levels that label each email as spam or email.
- `nonspam` will be the reference level, $Y = 0$,

```
is.factor(spam$type)
```

```
[1] TRUE
```

```
levels(spam$type)
```

```
[1] "nonspam" "spam"
```

- test-train splitting of the data

```
set.seed(42)
```

```
spam_idx <- sample(nrow(spam), 2000)
```

```
spam_trn <- spam[spam_idx,]
```

```
spam_tst <- spam[-spam_idx,]
```

spam Example III

- We fit four logistic regressions, each more complex than the previous. Note that we're suppressing two warnings.

```
fit_caps <- glm(type ~ capitalTotal,  
  data <- spam_trn, family = binomial)  
fit_selected <- glm(type ~ edu + money + capitalTotal + charDollar,  
  data = spam_trn, family = binomial)  
fit_additive <- glm(type ~ .,  
  data = spam_trn, family = binomial)  
fit_over <- glm(type ~ capitalTotal*(.),  
  data = spam_trn, family = binomial, maxit = 120)
```

- When we receive this warning, we should be highly suspicious of the parameter estimates.

```
message("Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred")
```

- However, the model can still be used to create a classifier, and we will evaluate that classifier on its own merits.
- We also, "suppressed" the warning:

```
message("Warning: glm.fit: algorithm did not converge")
```

spam Example IV

- In reality, we didn't actually suppress it, but instead changed 'maxit' to '120', when fitting the model `fit_over` to allow the iteratively reweighted least squares algorithm to converge when fitting the model.

Evaluating Classifiers I

- The metric we'll be most interested in for evaluating the overall performance of a classifier is the **misclassification rate**.

$$\text{Misclass}(\hat{C}, \text{Data}) = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{C}(x_{i,1}, \dots, x_{i,p-1}))$$

$$I(y_i \neq \hat{C}(x_{i,1}, \dots, x_{i,p-1})) = \begin{cases} 0 & y_i = \hat{C}(x_{i,1}, \dots, x_{i,p-1}) \\ 1 & y_i \neq \hat{C}(x_{i,1}, \dots, x_{i,p-1}) \end{cases}$$

- When using this metric on the training data, it will have the same issues as RSS did for ordinary linear regression, that is, it will only go down.

Evaluating Classifiers II

```
# training misclassification rate
mean(ifelse(predict(fit_caps) > 0, "spam", "nonspam") != spam_trn$type)
[1] 0.3515

mean(ifelse(predict(fit_selected) > 0, "spam", "nonspam") != spam_trn$type)
[1] 0.2015

mean(ifelse(predict(fit_additive) > 0, "spam", "nonspam") != spam_trn$type)
[1] 0.0735

mean(ifelse(predict(fit_over) > 0, "spam", "nonspam") != spam_trn$type)
[1] 0.0525
```

- Because of this, training data isn't useful for evaluating, as it would suggest that we should always use the largest possible model, when in reality, that model is likely overfitting.
- Recall, a model that is too complex will overfit. A model that is too simple will underfit. (We're looking for something in the middle.)
- To overcome this, we'll use cross-validation but this time we'll cross-validate the misclassification rate.

Evaluating Classifiers III

- We'll use the `cv.glm` function from the `boot` library.
- Arguments
 - the data (in this case training),
 - a model fit via `glm`, K the number of folds.

See `?cv.glm` for details.

- Essentially we'll repeat the following process 5 times:
 - Randomly set aside a fifth of the data (each observation will only be held-out once)
 - Train model on remaining data
 - Evaluate misclassification rate on held-out data
- The 5-fold cross-validated misclassification rate will be the average of these misclassification rates. By only needing to refit the model 5 times, instead of n times, we will save a lot of computation time.

Evaluating Classifiers IV

```
library(boot)
set.seed(1)
cv.glm(spam_trn, fit_caps, K = 5)$delta[1]
[1] 0.2184288
cv.glm(spam_trn, fit_selected, K = 5)$delta[1]
[1] 0.1519643
cv.glm(spam_trn, fit_additive, K = 5)$delta[1]
[1] 0.06775837
cv.glm(spam_trn, fit_over, K = 5)$delta[1]
[1] 0.102255
```

- Based on these results, `fit_caps` and `fit_selected` are underfitting relative to `fit_additive`. Similarly, `fit_over` is overfitting relative to `fit_additive`. Thus, based on these results, we prefer the classifier created based on the logistic regression fit and stored in `fit_additive`.
- Going forward, to evaluate and report on the efficacy of this classifier, we'll use the test dataset.

Evaluating Classifiers V

- To quickly summarize how well this classifier works, we'll create a confusion matrix.

		Actual	
		False (0)	True (1)
Predicted	False (0)	True Negative (TN)	False Negative (FN)
	True (1)	False Positive (FP)	True Positive (TP)

It further breaks down the classification errors into false positives and false negatives. We write an R function to create the confusion matrix:

```
make_conf_mat <- function(predicted, actual) {  
  table(predicted = predicted, actual = actual)  
}
```

Evaluating Classifiers VI

- Let's explicitly store the predicted values of our classifier on the test dataset.

```
spam_tst_pred <- ifelse(  
  predict(fit_additive, spam_tst) > 0,  
  "spam", "nonspam")  
spam_tst_pred <- ifelse(  
  predict(fit_additive, spam_tst, type = "response") > 0.5,  
  "spam", "nonspam")
```

- The previous two lines of code produce the same predictions, since

$$\beta_0 + \beta_1 x_1 + \dots + \beta_{p-1} x_{p-1} = 0 \iff p(x_1, \dots, x_{p-1}) = 0.5$$

Evaluating Classifiers VII

- We create a confusion matrix.

```
conf_mat_50 <- make_conf_mat(predicted = spam_tst_pred,  
  actual = spam_tst$type); conf_mat_50
```

	actual	
predicted	nonspam	spam
nonspam	1526	111
spam	82	882

- We have a *Prevalence* (total number of cases) of:

$$\text{Prev} = \frac{P}{\text{Total Obs}} = \frac{TP + FN}{\text{Total Obs}}$$

```
table(spam_tst$type) / nrow(spam_tst)
```

Evaluating Classifiers VIII

```
nonspam    spam  
0.6182238  0.3817762
```

- First, note that for a classifier to be reasonable, it needs to outperform the obvious classifier of simply classifying all observations to the majority class.
- In this case, classifying everything as non-spam for a test misclassification rate of 0.382
- Next, we can see that using the classifier create from `fit_additive`, only a total of $111 + 82 = 193$ from the total of 2601 email in the test set are misclassified. Overall, the accuracy in the test set is

```
mean(spam_tst_pred == spam_tst$type)  
[1] 0.9257978
```

In other words, the test set misclassification is

```
mean(spam_tst_pred != spam_tst$type)  
[1] 0.07420223
```

This seems like a decent classifier...

Evaluating Classifiers IX

- However, are all errors created equal? In this case, absolutely not.
- 82 non-spam emails marked as spam (false positives) are a problem.
- On the other hand, 111 spam email that would make it to an inbox (false negatives) are easily dealt with.
- Beside misclassification rate (or accuracy), we'll define two additional metrics: sensitivity and specificity.
- (Many!) other metrics that can be considered.

Sensitivity and Specificity I

Sensitivity

- Sensitivity is essentially the true positive rate. So when sensitivity is high, the number of false negatives is low.

$$\text{Sens} = \text{True Positive Rate} = \frac{\text{TP}}{\text{P}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Here we have an R function to calculate the sensitivity based on the confusion matrix.

- R code

```
get_sens <- function(conf_mat) {  
  conf_mat[2, 2] / sum(conf_mat[, 2])  
}
```

Sensitivity and Specificity II

Specificity

- Specificity is essentially the true negative rate. So when specificity is high, the number of false positives is low.

$$\text{Spec} = \text{True Negative Rate} = \frac{\text{TN}}{\text{N}} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

- R code

```
get_spec <- function(conf_mat) {  
  conf_mat[1, 1] / sum(conf_mat[, 1])  
}
```

Sensitivity and Specificity III

- We calculate both based on the confusion matrix we had created for our classifier.

```
get_sens(conf_mat_50) # true positive rate
```

```
[1] 0.8882175
```

```
get_spec(conf_mat_50) # true negative rate
```

```
[1] 0.949005
```

- We had created this classifier using a probability of 0.5 as a "cutoff"
- We can modify this cutoff and improve sensitivity or specificity
- The price to pay? The overall accuracy (misclassification rate)

$$\hat{C}(x_1, \dots, x_{p-1}) = \begin{cases} 1 & \hat{p}(x_1, \dots, x_{p-1}) > c \\ 0 & \hat{p}(x_1, \dots, x_{p-1}) \leq c \end{cases}$$

- Additionally, if we change the cutoff to improve sensitivity, we'll decrease specificity, and vice versa.

Sensitivity and Specificity IV

- First let's see what happens when we lower the cutoff from 0.5 to 0.1 to create a new classifier, and thus new predictions.

```
spam_tst_pred_10 = ifelse(predict(fit_additive, spam_tst,  
                                type = "response") > 0.1, "spam", "nonspam")
```

- This is essentially *decreasing* the threshold for an email to be labeled as spam: *more* emails will be labeled as spam (as seen in the confusion matrix)

```
conf_mat_10<-make_conf_mat(predicted = spam_tst_pred_10,  
                             actual = spam_tst$type); conf_mat_10
```

	actual	
predicted	nonspam	spam
nonspam	1191	19
spam	417	974

Sensitivity and Specificity V

Unfortunately, while this does greatly reduce false negatives, false positives have increased spectacularly. We see this reflected in the sensitivity and specificity.

```
get_sens(conf_mat_10) # true positive rate
```

```
[1] 0.9808661
```

```
get_spec(conf_mat_10) # true negative rate
```

```
[1] 0.7406716
```

```
# we had
```

```
c(get_sens(conf_mat_50), get_spec(conf_mat_50))
```

```
[1] 0.8882175 0.9490050
```

Sensitivity and Specificity VI

- This classifier, using 0.1 instead of 0.5 has a higher sensitivity, but a much lower specificity. Clearly, we should have moved the cutoff in the other direction. Let's try 0.9.

```
spam_tst_pred_90 <- ifelse(predict(fit_additive,spam_tst,  
                                type = "response") > 0.9, "spam", "nonspam")
```

This is essentially *increasing* the threshold for an email to be labeled as spam, so far *fewer* emails will be labeled as spam. Again, we see that in the confusion matrix.

```
conf_mat_90 <- make_conf_mat(predicted = spam_tst_pred_90,  
                              actual = spam_tst$type); conf_mat_90
```

	actual	
predicted	nonspam	spam
nonspam	1589	373
spam	19	620

Sensitivity and Specificity VII

- This is the result we're looking for. We have far fewer false positives. While sensitivity is greatly reduced, specificity has gone up.

```
get_sens(conf_mat_90) # true positive rate
```

```
[1] 0.6243706
```

```
get_spec(conf_mat_90) # true negative rate
```

```
[1] 0.9881841
```

```
# we had
```

```
c(get_sens(conf_mat_50), get_spec(conf_mat_50))
```

```
[1] 0.8882175 0.9490050
```

```
c(get_sens(conf_mat_10), get_spec(conf_mat_10))
```

```
[1] 0.9808661 0.7406716
```

Table of content I

- 1 GLM
 - Inference for GLM

- 2 Classification